



CS3014 - Estructuras de Datos Avanzadas

Notas de clase 01

Strings

31 de mayo
de 2026

Prof. Víctor Racsó Galván Oyola

1. Notación

Sea Σ un conjunto ordenado de símbolos (a partir de ahora, *caracteres*), el cual llamaremos *alfabeto*. Diremos que una secuencia finita S que consiste de caracteres de Σ es una *cadena* sobre el alfabeto Σ . Por convención, denotaremos la cadena vacía como ε . Además, denotaremos la longitud de S por $|S|$, el i -ésimo carácter ($0 \leq i < |S|$) de S por $S[i]$ y una subcadena que consiste de la concatenación de los caracteres de S desde la posición i hasta la posición j por $S[i, j]$ para cualquier par (i, j) tal que $0 \leq i \leq j < |S|$. Denotaremos por $\text{Sub}(S)$ al conjunto de todas las subcadenas de S .

Definición 1 (Prefijo y sufijo de una cadena)

El *prefijo* de una cadena S que termina en la posición i es la subcadena $S[0, i]$ y será denotada por $\text{Pref}(S, i)$.

El *sufijo* de una cadena S que comienza en la posición i es la subcadena $S[i, |S| - 1]$ y será denotada por $\text{Suf}(S, i)$.

Un prefijo o sufijo de S es llamado **propio** si su longitud es **estrictamente menor** que la de S .

La concatenación de las cadenas U y V está denotada por $U \cdot V$ o UV . La misma notación se extiende para concatenación de carácter-cadena y carácter-carácter. Para un entero $k \geq 1$, denotamos la concatenación de k copias de una cadena U como U^k . Además, la concatenación de las cadenas $S_1, S_2, \dots, S_m$ se denota como $\prod_{i=1}^m S_i$.

Denotaremos el conjunto que contiene todas las cadenas de longitud finita sobre el alfabeto Σ como Σ^* .

Dadas dos cadenas S y T , su *prefijo en común más largo* (LCP) es la cadena $P \in \Sigma^*$ más larga tal que $S = P \cdot U$ y $T = P \cdot V$ para algún par de cadenas $U, V \in \Sigma^*$. Lo denotaremos por $\text{LCP}(S, T) = P$.

Asumiremos que el alfabeto Σ contiene un carácter llamado **sentinela** (\$) que precede a todos los demás símbolos de Σ y ocurre exclusivamente al final de cada cadena de entrada. Tener un sentinela simplifica la comparación de sufijos debido a que evita el tener cadenas vacías.

Definición 2 Orden lexicográfico

Sea $<$ el orden total sobre Σ inducido por el orden de sus elementos. Este orden puede ser extendido a las cadenas $S, T \in \Sigma^*$. Diremos que S es lexicográficamente menor que T si se cumple alguna de las siguientes dos condiciones:

- S es un prefijo propio de T .

- S y T difieren en una posición. Sea p la menor posición en la que difieren, entonces $S[p] \prec T[p]$.

Por convención, usaremos comparación lexicográfica al comparar dos cadenas.

2. Suffix Array

El suffix array es una estructura de indexing cuya característica principal es la poca cantidad de memoria que usa a comparación de las demás estructuras. Consideraremos que tenemos una cadena de entrada S con longitud n . Esta se define de la siguiente manera:

Definición 1 (Suffix Array)

Dada una cadena S de longitud n , el suffix array asociado a S es la permutación A de tamaño n tal que:

$$\text{Suf}(S, A_i) \prec \text{Suf}(S, A_{i+1}), \forall i = 0, \dots, (n-2)$$

Por lo tanto, requeriremos exactamente n enteros para esta estructura, lo cual es una ventaja en muchos escenarios.

2.1. Manber y Myers

Manber y Myers fueron quienes propusieron el suffix array por primera vez en [3]; además diseñaron un algoritmo para su construcción en tiempo $O(n \log n)$.

La idea para la construcción del suffix array en $O(n \log n)$ podría decirse que está basada en los *Sparse Tables*, así que primero revisaremos un enfoque que nos permita comparar subcadenas de S de manera eficiente.

Consideremos que existe una función $\sigma : \text{Sub}(S) \rightarrow \mathbb{R}$ que mapee cada subcadena diferente de S a un real distinto, entonces es sencillo notar que requeriremos almacenar a lo mucho $O(n^2)$ reales diferentes para ello, lo cual no es posible para valores muy grandes de n .

Si decidimos particionar $\text{Sub}(S)$ por longitud ℓ de cada subcadena (denotemos esto por $\text{Sub}_\ell(S)$), requeriremos $O(n)$ reales diferentes para un mapeo $\sigma_\ell : \text{Sub}_\ell(S) \rightarrow \mathbb{R}$, lo cual sí se puede almacenar. Para no requerir la misma cantidad de memoria que el escenario de σ , vamos a seleccionar solo un conjunto reducido de ℓ que nos permita comparar subcadenas en tiempo eficiente.

Lema 2

Dada la cadena S de longitud n , basta con elegir el subconjunto $\mathcal{L} = \{2^k : 1 \leq 2^k \leq n\}$ y los mapeos σ_ℓ para todo $\ell \in \mathcal{L}$ para poder comparar subcadenas en $O(1)$ con un preprocesamiento de $O(n \log n)$.

El proceso sería el siguiente:

Algoritmo 1: CONSTRUIR-MAPEOS(S, n)

```

1  $k \leftarrow 1 + \lfloor \log_2 n \rfloor$  ;
2 Sea  $\text{mapeo}[k][n]$  un arreglo de enteros ;
3  $p \leftarrow \{0, \dots, n-1\}$  ;
4 Ordenar  $p$  por  $S_i$  creciente ;
5 Asignar  $\text{mapeo}[0]$  de acuerdo a  $p$  ;
6 para  $d \leftarrow 1$  a  $k-1$  hacer
7    $v \leftarrow \{(\text{mapeo}[d-1][i], \text{mapeo}[d-1][i+2^{d-1}]) : 0 \leq i \leq n-2^{d-1}\}$  ;
8   Filtrar  $p$  para quedarnos con solo los  $i \in [0, n-2^d]$  ;
9   Ordenar  $p$  por  $v$  creciente usando Radix sort ;
10  Asignar  $\text{mapeo}[d]$  de acuerdo a  $p$  ;
11 devolver  $\text{mapeo}$ 

```

Sin embargo, es posible realizar una pequeña mejora para no necesitar usar Radix sort, para ello debemos realizar la siguiente observación:

Observación 3

El estado de p al llegar a la iteración d ya contiene los i de dicho nivel ordenados por su segunda componente en v .

La observación anterior nos da la idea de que podemos iterar sobre p al momento de filtrar los i del nivel y **solo filtrar aquellos que sean mayores o iguales que 2^{d-1}** y colocarlos como $i - 2^{d-1}$ en su posición correcta. Luego de ello, ordenaremos p por el valor de $\text{mapeo}[d-1][i]$ de manera **estable** (usando counting sort).

Algoritmo 2: CONSTRUIR-MAPEOS(S, n)

```

1  $k \leftarrow 1 + \lfloor \log_2 n \rfloor$  ;
2 Sea  $\text{mapeo}[k][n]$  un arreglo de enteros ;
3  $p \leftarrow \{1, \dots, n\}$  ;
4 Ordenar  $p$  por  $S_i$  creciente ;
5 Asignar  $\text{mapeo}[0]$  de acuerdo a  $p$  ;
6 para  $d \leftarrow 1$  a  $k-1$  hacer
7   Filtrar  $p$  para quedarnos con solo los  $i \in [2^{d-1}, n-2^{d-1}]$  mapeados a  $i - 2^{d-1}$  ;
8   Ordenar  $p$  por  $\text{mapeo}[d-1][i]$  creciente de manera estable (usando Counting sort) ;
9   Asignar  $\text{mapeo}[d]$  de acuerdo a  $p$  ;
10 devolver  $\text{mapeo}$ 

```

Veamos un ejemplo para visualizar bien el proceso:

Ejemplo 4

Sea

$$S = \text{mississippi}, \quad n = 11, \quad k = 4.$$

Luego de ordenar los sufijos según S_i obtenemos:

$$p = (1, 4, 7, 10, 0, 8, 9, 2, 3, 5, 6).$$

Nivel $d = 0$

El mapeo inicial queda dado por:

i	0	1	2	3	4	5	6	7	8	9	10
$\text{mapeo}[0]$	1	0	3	3	0	3	3	0	2	2	0

Nivel $d = 1$

Se filtra el vector p y mapeamos $i \mapsto i - 2^{d-1} = i - 1$:

$$(1, 4, 7, 10, 8, 9, 2, 3, 5, 6) \longrightarrow (0, 3, 6, 9, 7, 8, 1, 2, 4, 5).$$

Luego de ordenar de manera estable según $\text{mapeo}[0]$, se obtiene:

$$p = (7, 1, 4, 0, 9, 8, 3, 6, 2, 5).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10
$\text{mapeo}[1]$	2	1	6	5	1	6	5	0	4	3	?

Nivel $d = 2$

Se filtra p y mapeamos $i \mapsto i - 2^{d-1} = i - 2$:

$$(7, 4, 9, 8, 3, 6, 2, 5) \longrightarrow (5, 2, 7, 6, 1, 4, 0, 3).$$

Luego de ordenar de manera estable según $\text{mapeo}[1]$, se obtiene:

$$p = (7, 1, 4, 0, 6, 3, 5, 2).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10
$\text{mapeo}[2]$	2	1	6	4	1	5	3	0	?	?	?

Nivel $d = 3$

Se filtra p y mapeamos $i \mapsto i - 2^{d-1} = i - 4$:

$$(7, 4, 6, 5) \longrightarrow (3, 0, 2, 1).$$

Luego de ordenar de manera estable según $\text{mapeo}[2]$, se obtiene:

$$p = (1, 0, 3, 2).$$

El mapeo final es:

i	0	1	2	3	4	5	6	7	8	9	10
$\text{mapeo}[3]$	1	0	3	2	?	?	?	?	?	?	?

Entonces, es posible ordenar todos los sufijos de S en $O(n \log n)$ si usamos la función **compare** como subrutina para **std::sort** manejando adecuadamente las longitudes diferentes en los sufijos.

A pesar de que el algoritmo anterior nos permite construir el suffix array en $O(n \log n)$, requiere $O(n \log n)$ de memoria, lo cual no es tan bueno para la eficiencia.

Ahora, sería muy bueno para nosotros si consiguiéramos definir σ_{2^m} con $2^m \geq n$ y mínimo posible, ya que con dichos mapeos podremos comparar absolutamente todos los sufijos entre sí (bastaría con comparar σ_{2^m} de cada sufijo).

Ya que no podemos extender la cadena arbitrariamente esperando que algo así funcione, no lo

haremos ... ¿o sí? En realidad es posible extender S usando $2^m - 1$ caracteres sentinelas $\$$. Esta última condición es necesaria para representar cadenas que originalmente no estarían completas (en cuyo caso, la relación cuando una es prefijo de otra se establece correctamente ya que $\$$ precede a cualquier $c \in \Sigma$), así que el caracter $\$$ funciona como un separador.

De esta manera, cuando construyamos los mapeos, podemos simplemente quedarnos con el nivel anterior y al final de toda la ejecución filtrar las posiciones originales de S . Esto nos reduce la memoria usada a $O(n)$, pero con una constante bastante alta.

Para reducir la memoria usada, podemos aprovechar la propiedad del sentinela como menor lexicográfico que todo el alfabeto, de manera que en vez de agregar $2^m - 1$ sentinelas, agregaremos solo uno pero consideraremos las subcadenas como **cíclicas** cuando calculemos σ_{2^m} .

Lo interesante de esta idea es que podemos mantener siempre todas las posiciones de $S \cdot \$$ e ir calculando los mapeos y ordenamientos de ellas nivel por nivel. Ya que cada nivel depende del anterior, no necesitamos almacenar todos los niveles, pues las posiciones finales ordenadas coincidirán con el suffix array de $S \cdot \$$.

Una observación extra que se puede considerar es que, así como *mapeo* tiene los mapeos del nivel correspondiente, las posiciones están ordenadas por dichos mapeos, así que podemos asignar las posiciones iniciales de cada bucket del counting sort simplemente iterando de mayor a menor índice sobre las posiciones y “machacando” los inicios de los buckets. Esto nos permitiría hacer el ordenamiento de las posiciones según la nueva longitud en solo 3 iteraciones.

Usaremos el siguiente pseudocódigo para obtener el suffix array:

Algoritmo 3: MANBER-MYERS(S)

```

1   $n \leftarrow |S|$   $a \leftarrow \{0, \dots, n-1\}$  ;
2  Ordenar  $a$  según  $S_i$  creciente ;
3  Sean  $mapeo[n]$ ,  $new\_mapeo[n]$ ,  $head[n]$  y  $by\_second[n]$  arreglos de enteros ;
4   $len \leftarrow 1$  ;
5  mientras  $len < n$  hacer
6      para  $i \leftarrow 0$  a  $n-1$  hacer
7           $by\_second[i] \leftarrow (a[i] - len + n) \pmod n$  ;
8      para  $i \leftarrow n-1$  a  $0$  hacer
9           $head[mapeo[a[i]]] \leftarrow i$  ;
10     para  $i \leftarrow 0$  a  $n-1$  hacer
11          $x \leftarrow by\_second[i]$  ;
12          $a[head[mapeo[x]]] \leftarrow x$  ;
13          $head[mapeo[x]] \leftarrow head[mapeo[x]] + 1$  ;
14     Asignar en  $new\_mapeo$  los mapeos correspondientes a  $a$  ;
15     Intercambiar  $mapeo$  con  $new\_mapeo$  ;
16      $len \leftarrow 2 \times len$  ;
17 devolver  $a$  ;
```

Veamos un ejemplo para visualizar bien el proceso:

Ejemplo 5

Sea

$$S = \text{mississippi}\$, \quad n = 12.$$

Luego de ordenar los sufijos según S_i obtenemos:

$$a = (11, 1, 4, 7, 10, 0, 8, 9, 2, 3, 5, 6).$$

Iteración $len = 1$

El mapeo inicial queda dado por:

i	0	1	2	3	4	5	6	7	8	9	10	11
$mapeo$	2	1	4	4	1	4	4	1	3	3	1	0

Iteración $len = 2$

Luego de asignar el vector by_second , este queda de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
by_second	2	1	4	4	1	4	4	1	3	3	1	0

Y los $head$ quedarían de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
$head$	0	1	5	6	8	0	0	0	0	0	0	0

Luego de ejecutar la líneas 10-13 del algoritmo, se obtiene:

$$a = (11, 10, 7, 1, 4, 0, 9, 8, 3, 6, 2, 5).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10	11
$mapeo$	4	3	8	7	3	8	7	2	6	5	1	0

Iteración $len = 4$

Luego de asignar el vector by_second , este queda de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
by_second	9	8	5	11	2	10	7	6	1	4	0	3

Y los $head$ quedarían de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
$head$	0	1	2	3	5	6	7	8	10	0	0	0

Luego de ejecutar la líneas 10-13 del algoritmo, se obtiene:

$$a = (11, 10, 7, 1, 4, 0, 9, 8, 6, 3, 5, 2).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10	11
$mapeo$	4	3	10	8	3	9	7	2	6	5	1	0

Iteración $len = 8$

Luego de asignar el vector by_second , este queda de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
by_second	7	6	3	9	0	8	5	4	2	11	1	10

Y los $head$ quedarían de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
$head$	0	1	2	3	5	6	7	8	9	10	11	0

Luego de ejecutar la líneas 10-13 del algoritmo, se obtiene:

$$a = (11, 10, 7, 4, 1, 0, 9, 8, 6, 3, 5, 2).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10	11
$mapeo$	5	4	11	9	3	10	8	2	7	6	1	0

Iteración $len = 16$

Luego de asignar el vector by_second , este queda de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
by_second	3	2	11	8	5	4	1	0	10	7	9	6

Y los $head$ quedarían de la siguiente manera:

i	0	1	2	3	4	5	6	7	8	9	10	11
$head$	0	1	2	3	4	5	6	7	8	9	10	11

Luego de ejecutar la líneas 10-13 del algoritmo, se obtiene:

$$a = (11, 10, 7, 4, 1, 0, 9, 8, 6, 3, 5, 2).$$

El nuevo mapeo es:

i	0	1	2	3	4	5	6	7	8	9	10	11
$mapeo$	5	4	11	9	3	10	8	2	7	6	1	0

Y esta idea es más sencilla a nivel de implementación.

2.2. Karkkainen, Sanders y Burkhardt

A pesar de que Ko y Aluru fueron los primeros en proponer un algoritmo lineal [2] para la construcción del suffix array, su código no es práctico (el original tenía más de 1000 líneas), así que presentaremos primero el algoritmo DC3 (*Difference Cover* módulo 3) [1].

La idea detrás del algoritmo se basa en aplicar recursión para reducir S a una cadena R de menor longitud, calcular el suffix array de R para obtener el suffix array de S a partir de dicha información. Notemos que $n = 1$ es el caso base de la recursión y su respuesta siempre es $A = \{0\}$.

El algoritmo propone los siguientes pasos:

1. Definir los conjuntos de posiciones B_0 , B_1 y B_2 , donde

$$B_k = \{0 \leq i \leq n - 1 : i \equiv k \pmod{3}\}$$

Declaramos $C = B_1 \cup B_2$ y denotamos por S_C el conjunto de sufijos de C .

Consideremos que si $n \equiv 1 \pmod{3}$, entonces se agrega n a B_1 para asegurar la presencia de sentinelas como separadores.

2. Para $k = 1, 2$, consideraremos la cadena formada de la siguiente manera:

$R_k = \text{Suf}(S, k)$ agrupado en grupos de 3 caracteres consecutivos

Consideremos que si algún grupo contiene menos de 3 caracteres, completaremos dichos caracteres con sentinelas \$.

Por ejemplo, para $S = \text{mississippi}$ con $n = 11$, se daría que

$$\begin{aligned} R_1 &= (\text{iss})(\text{iss})(\text{ipp})(\text{i}\$ \$) \\ R_2 &= (\text{ssi})(\text{ssi})(\text{ppi}) \end{aligned}$$

Además, se define $R = R_1 \cdot R_2$, es decir

$$R = (\text{iss})(\text{iss})(\text{ipp})(\text{i}\$ \$)(\text{ssi})(\text{ssi})(\text{ppi})$$

Luego, asignaremos mapeo numérico a cada terna de caracteres en función a su posición lexicográfica, reduciendo R a una secuencia de enteros S' :

$$\begin{aligned} \text{i}\$ \$ &\Rightarrow 0 \\ \text{ipp} &\Rightarrow 1 \\ \text{iss} &\Rightarrow 2 \\ \text{ppi} &\Rightarrow 3 \\ \text{ssi} &\Rightarrow 4 \end{aligned}$$

$$S' = 2210443$$

Si todos los elementos de S' son diferentes, con su mapeo numérico podemos obtener el orden de sus sufijos; en caso contrario, podemos aplicar recursión sobre S' para obtener el orden de sus sufijos. Con el orden, podemos asignar un rango (posición ordenada) $\text{rank}_S(i)$ a cada sufijo. Notemos que la comparación de sufijos de S' va a preservar el orden de cada sufijo en B_1 y B_2 debido a los sentinelas separadores.

En este ejemplo, tendríamos

S'_i	2	2	1	0	4	4	3
$\text{rank}_{S'}(i)$	3	2	1	0	6	5	4

Lo cual se traduce a

S_i	m	i	s	s	i	s	s	i	p	p	i
$\text{rank}_S(i)$	?	3	6	?	2	5	?	1	4	?	0

3. Para ordenar los sufijos de B_0 entre sí, podemos representar cada posición i como un par $(S_i, \text{rank}_S(i+1))$ y usar radix sort.

Para el ejemplo mostrado, tendríamos

$$(\mathbf{m}, 3) \prec (\mathbf{p}, 0) \prec (\mathbf{s}, 1) \prec (\mathbf{s}, 2)$$

$$\text{Suf}_S(0) \prec \text{Suf}_S(9) \prec \text{Suf}_S(6) \prec \text{Suf}_S(3)$$

4. Ahora tendremos los dos conjuntos de sufijos ordenados, los de B_0 y los de C . Para ordenarlos entre sí podemos plantear el siguiente método de comparación para los dos posibles escenarios:

- $i \in B_0$ y $j \in B_1$: Comparamos por pares como en B_0 .
- $i \in B_0$ y $j \in B_2$: Comparamos por ternas $(S_i, S_{i+1}, \text{rank}_S(i+2))$ y $(S_j, S_{j+1}, \text{rank}_S(j+2))$.

Asumiremos que tendremos una función llamada $\text{COMPARE-DC3}(i, j, S, \text{rnk})$ que permita comparar las posiciones i y j según este criterio (asumiremos que devuelve una respuesta en $[-1, 1]$ como un comparador clásico de C).

Adaptando el clásico MERGE que se usa en el algoritmo de Merge Sort, obtendremos los sufijos ordenados en $O(n)$.

Algoritmo 4: MERGE-DC3(B_0, A_R, S, rnk)

```

1  $A \leftarrow \emptyset$  ;
2  $\text{pos}_C \leftarrow 0$  ;
3 para  $i \in B_0$  hacer
4   mientras  $\text{pos}_C < |A_R|$  y  $\text{COMPARE-DC3}(i, A_R[\text{pos}_C], S, \text{rnk}) = 1$  hacer
5      $A \leftarrow A \cup \{A_R[\text{pos}_C]\}$  ;
6      $\text{pos}_C \leftarrow \text{pos}_C + 1$  ;
7    $A \leftarrow A \cup \{i\}$  ;
8 mientras  $\text{pos}_C < |A_R|$  hacer
9    $A \leftarrow A \cup \{A_R[\text{pos}_C]\}$  ;
10   $\text{pos}_C \leftarrow \text{pos}_C + 1$  ;
11 devolver  $A$ 
```

La complejidad del algoritmo se basa en la siguiente recursión:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Ya que la cadena S' tendrá un tamaño que no excede a $\frac{2n}{3}$ y todos los demás pasos toman $O(n)$. Es sencillo notar que la recursión se resuelve con

$$T(n) = O(n)$$

Así que tendremos un algoritmo lineal.

Algoritmo 5: DC3(n, S)

```

1 Sean  $B_k = \{0 \leq i < n : i \equiv k \pmod{3}\}$  para  $k = 0, 1, 2$  ;
2 si  $n \equiv 1 \pmod{3}$  entonces
3    $B_1 \leftarrow B_1 \cup \{n\}$  ;
4  $C \leftarrow B_1 \cup B_2$  ;
5 Ordenar  $C$  por  $(S[x], S[x+1], S[x+2])$  con RADIX-SORT ;
6 Crear  $\sigma : C \mapsto \mathbb{Z}_0^+$  según el orden de  $C$  ;
7  $R \leftarrow \prod_{i=1}^2 \prod_{x \in B_i} \sigma((S[x], S[x+1], S[x+2]))$  ;
8  $A_R \leftarrow \text{DC3}(|R|, R)$  ;
9 Sea  $rnk[0 \dots n]$  un arreglo de enteros ;
10 para  $i \leftarrow 0$  a  $|R| - 1$  hacer
11   si  $A_R[i] < |B_1|$  entonces
12      $x \leftarrow B_1[A_R[i]]$ 
13   en otro caso
14      $x \leftarrow B_2[A_R[i] - |B_1|]$ 
15    $rnk[x] \leftarrow i$ 
16 Ordenar  $B_0$  por  $(S[x], rnk[x+1])$  con RADIX-SORT ;
17 devolver MERGE-DC3( $B_0, A_R, S, rnk$ )

```

Donde los accesos a posiciones que no existen se deben considerar como -1 en valor.

2.3. Ko y Aluru (2003)

En [2], Ko y Aluru presentan, por primera vez, un algoritmo lineal que aprovecha la clasificación de sufijos de una cadena S .

Definición 6 (Clasificación de sufijos)

Dada una cadena S , el sufijo $\text{Suf}(S, i)$ es clasificado en uno de los dos siguientes tipos:

- Tipo L, si $\text{Suf}(S, i) > \text{Suf}(S, i+1)$ (en caso exista).
- Tipo S, si $\text{Suf}(S, i) < \text{Suf}(S, i+1)$ (en caso exista).

El sufijo $\text{Suf}(S, n-1)$ puede ser considerado de tipo L o S según convenga.

Es posible obtener la clasificación de cada uno de los n sufijos en tiempo $O(n)$ mediante la siguiente idea:

- $T[n-1] = \text{S}$ o $T[n-1] = \text{L}$ según convenga.
- Si $S[i] = S[i+1]$, entonces $T[i] \leftarrow T[i]$.
- En caso contrario, entonces $T[i]$ se define según la relación entre $S[i]$ y $S[i+1]$.

El algoritmo sería el siguiente:

Algoritmo 6: $\text{CLASSIFY}(n, S)$

```

1 Sea  $T[0 \dots (n-1)]$  un arreglo de tipos ;
2  $T[n-1] \leftarrow S$  ;
3 para  $i \leftarrow n-2$  a 0 hacer
4   si  $S[i] = S[i+1]$  entonces
5      $T[i] \leftarrow T[i+1]$  ;
6   en otro caso
7     si  $S[i] < S[i+1]$  entonces
8        $T[i] \leftarrow S$  ;
9     en otro caso
10       $T[i] \leftarrow L$  ;
11 devolver  $T$  ;

```

Debido a la definición de los tipos, se puede probar el siguiente lema:

Lema 7

Sea c el caracter con el que comienza $\text{Suf}(S, i)$. Si $T[i] = S$, entonces $\text{Suf}(S, i) > ccc \dots$; mientras que si $T[i] = L$, entonces $\text{Suf}(S, i) < ccc \dots$

Como un corolario del teorema anterior podemos concluir que:

Corolario 8

Cualquier sufijo de tipo S que comience con el caracter c es mayor lexicográficamente que cualquier sufijo de tipo L que comience con el mismo caracter.

Con respecto a las posiciones de los sufijos que comienzan con un mismo caracter c , también se obtiene el siguiente corolario:

Corolario 9

Todos los sufijos de tipo S que comienzan con el caracter c van después de todos los sufijos de tipo L que también comienzan con c en el suffix array.

Bajo estas condiciones sobre los tipos, una acción natural sería agrupar los sufijos por su letra inicial (bloques) e intentar ordenar un subconjunto de ellos para obtener las posiciones de todos los sufijos de alguno de los tipos e inducir el orden de los del otro tipo.

Vamos a asumir que tenemos una subrutina que nos permita calcular los inicios y finales de cada bloque de sufijos que comience con la misma letra, esta estará basada en el algoritmo de counting sort y asumirá que todos los caracteres de S están en el rango $[0, n-1]$. Esta subrutina tendrá una complejidad de $O(n)$ bajo la restricción de los caracteres.

Supongamos que X es la secuencia ordenada lexicográficamente de los sufijos de tipo S de S , entonces podemos inducir el orden de todos los sufijos de S notando lo siguiente:

Observación 10

Si S_i es un sufijo de tipo L (S), entonces el sufijo S_{i+1} debe aparecer antes (después) que S_i en el suffix array.

Bajo la observación anterior, es posible ordenar todos los sufijos siguiendo los pasos a continuación:

1. Colocamos los sufijos de X en sus posiciones correspondientes en el suffix array A .

2. Iteramos de izquierda a derecha (derecha a izquierda) sobre los elementos de A , si $S_{A[i]-1}$ es de tipo L (S), entonces lo colocamos al frente (final) de su bloque y movemos el puntero correspondiente.

Es sencillo notar que la complejidad del algoritmo anterior es $O(n)$, ya que podemos mantener el inicio y el final de cada bloque (para solo sumar o restar 1 cuando asignemos algún sufijo a una posición) en dos arreglos para procesar cada iteración en $O(1)$.

Se pueden usar los siguientes algoritmos para ello.

Algoritmo 7: GET-BUCKETS(n, S)

```

1 Sea  $cnt[0 \dots (|\Sigma| - 1)]$  un arreglo de enteros inicializado en 0 ;
2 para  $i \leftarrow 0$  a  $n - 1$  hacer
3    $cnt[S[i]] \leftarrow cnt[S[i]] + 1$  ;
4 Sean  $L[0 \dots (|\Sigma| - 1)]$  y  $R[0 \dots (|\Sigma| - 1)]$  arreglos de enteros ;
5  $sum \leftarrow 0$  ;
6 para  $i \leftarrow 0$  a  $(|\Sigma| - 1)$  hacer
7    $sum \leftarrow sum + cnt[i]$  ;
8    $L[i] \leftarrow sum - cnt[i] + 1$  ;
9    $R[i] \leftarrow sum$  ;
10 devolver  $L, R$ 
```

Algoritmo 8: INDUCE-ORDER-WITH-S(n, S, T, X)

```

1  $L, R = \text{GET-BUCKETS}(n, S)$  ;
2 Sea  $A[0 \dots (n - 1)]$  un arreglo de enteros inicializado en  $-1$  ;
3 para  $i \leftarrow |X| - 1$  a  $0$  hacer
4    $c \leftarrow S[X[i]]$  ;
5    $A[R[c]] \leftarrow X[i]$  ;
6    $R[c] \leftarrow R[c] - 1$  ;
7 para  $i \leftarrow 0$  a  $n - 1$  hacer
8   si  $A[i] > 0$  y  $T[A[i] - 1] = L$  entonces
9      $c \leftarrow S[A[i] - 1]$  ;
10     $A[L[c]] \leftarrow A[i] - 1$  ;
11     $L[c] \leftarrow L[c] + 1$  ;
12 devolver  $A$ 
```

Ahora debemos intentar encontrar la manera de ordenar los sufijos de un tipo en particular de forma eficiente. Notemos que es posible particionar S tomando como referencia a sus sufijos de tipo S o L, definiendo las subcadenas de tipo S y L.

Definición 11 (Subcadena L/S)

Una subcadena $S[i, j]$ es de tipo x si y solo si:

- S_i y S_j son de tipo x .
- No hay ningún $i < k < j$ tal que S_k sea de tipo x .

Por ejemplo, para la cadena “bdefdfdfbdeddaf d ”\$, se tendrían los tipos y subcadenas:

	b	d	e	f	d	f	d	f	b	d	e	d	d	a	f	d	\$
Tipo	S	S	S	L	S	L	S	L	S	S	L	L	L	S	L	L	S/L

Cuadro 1: Tipos de sufijo de la cadena “bdefdfdfbdeddafdf\$”

Subcadenas S	Subcadenas L
▪ afd\$ ▪ bd ▪ de ▪ dedda ▪ dfb ▪ dfd ▪ efd	▪ d\$ ▪ daf ▪ dd ▪ ed ▪ fbde ▪ fd ▪ fdf

Cuadro 2: Subcadenas de la cadena “bdefdfdfbdeddafdf\$”

Si asociáramos cada subcadena a su posición de inicio, entonces se formarían las siguientes dos cadenas:

Subcadenas S	Subcadenas L
1. bd	1. fdf
2. de	2. fdf
3. efd	3. fbde
4. dfd	4. ed
5. dfb	5. dd
6. bd	6. daf
7. dedda	7. fd
8. afd\$	8. d\$

Cuadro 3: Partición por tipo de la cadena “bdefdfdfbdeddafdf\$”

Notemos que en el caso de usar el tipo L, asumimos que el caracter sentinela es de tipo L. Por otro lado, es posible que un prefijo de la cadena sea ignorado en el mapeo debido a que no es relevante para ordenar los sufijos del tipo deseado.

Ahora debemos plantear el mapeo que se usará en cada tipo. Para ello, definiremos el siguiente enfoque:

Enfoque 12

Si se consideran las subcadenas de tipo S (L), una subcadena X debe recibir un menor mapeo que Y si y solo si:

- Existe una posición j tal que $X_j \neq Y_j$ y $X_j \prec Y_j$.
- $Y(X)$ es prefijo propio de $X(Y)$.

A partir de este enfoque, se cumple el siguiente lema.

Lema 13

Sea S' la cadena compuesta por los mapeos de las subcadenas elegidas de S , entonces para dos sufijos S_i y S_j con sus posiciones correspondientes en $S'_{i'}$ y $S'_{j'}$, respectivamente:

$$S_i \prec S_j \iff S'_{i'} \prec S'_{j'}$$

Vamos a reemplazar cada inicio de subcadena por su mapeo correspondiente y dejaremos el sentinela en la última posición. Para el ejemplo mostrado, los dos mapeos serían:

Subcadenas S	Subcadenas L
1. bd $\rightarrow$ 1	1. fdf $\rightarrow$ 6
2. de $\rightarrow$ 3	2. fdf $\rightarrow$ 6
3. efd $\rightarrow$ 6	3. fbde $\rightarrow$ 4
4. dfd $\rightarrow$ 5	4. ed $\rightarrow$ 3
5. dfb $\rightarrow$ 4	5. dd $\rightarrow$ 2
6. bd $\rightarrow$ 1	6. daf $\rightarrow$ 1
7. dedda $\rightarrow$ 2	7. fd $\rightarrow$ 5
8. afd\$ $\rightarrow$ 0	8. d\$ $\rightarrow$ 0
9. \$ $\rightarrow$ \$	9. \$ $\rightarrow$ \$

Cuadro 4: Mapeo por tipo de la cadena “bdefdfdfbdeddaf\$”

Observemos que la comparación de los sufijos “defdfdfbdeddaf\$” y “deddaf\$” es correcta bajo el mapeo de las subcadenas S, así como también lo es la comparación de los sufijos “fdfbdeddaf\$” y “fd\$” bajo el mapeo de las subcadenas L.

Además, el orden de los sufijos mapeados de la cadena $S' = 13654120\$$ es $\{8, 7, 5, 0, 6, 1, 4, 3, 2\}$ y el orden de todos los sufijos de S (tipo S) correspondientes sería $\{13, 8, 0, 9, 1, 6, 4, 2\}$. Por otro lado, el orden de los sufijos mapeados de la cadena $S' = 66432150\$$ es $\{8, 7, 5, 4, 3, 2, 6, 1, 0\}$ y el orden de todos los sufijos de S (tipo L) correspondientes sería $\{13, 8, 0, 9, 1, 6, 4, 2\}$.

Una observación trivial pero importante es la siguiente:

Observación 14

El sufijo $S_{n-1} = \$$ siempre será el primero en el suffix array, así que se puede colocar directamente.

2.3.1. Funciones para calcular S'

Vamos a presentar los algoritmos que se requieren para obtener S' tomando como referencia a las subcadenas de tipo S. Para el escenario que usa subcadenas de tipo L es sencillo notar que

funciones deben modificarse.

Algoritmo 9: GROUP-BY-CHARACTER(n, S)

```

1 Sea  $A[n]$  un arreglo de enteros ;
2 Sea  $head[n]$  un arreglo de enteros inicializado en 0 ;
3 para  $i \leftarrow 0$  a  $n - 1$  hacer
4    $head[S[i]] \leftarrow head[S[i]] + 1$  ;
5  $sum \leftarrow 0$  ;
6 para  $i \leftarrow 0$  a  $n - 1$  hacer
7    $sum \leftarrow sum + head[i]$  ;
8    $head[i] \leftarrow sum - head[i]$  ;
9 para  $i \leftarrow 0$  a  $n - 1$  hacer
10   $A[head[S[i]]] \leftarrow i$  ;
11   $head[S[i]] \leftarrow head[S[i]] + 1$  ;
12 devolver  $A$  ;
```

Esta función nos devuelve las posiciones de A ordenadas por su primer caracter, desempataando arbitrariamente.

Los siguientes algoritmos son los principales para obtener el mapeo de las subcadenas elegidas para la partición de S .

Algoritmo 10: GET-S-DISTANCES(n, S, T)

```

1 Sea  $Dist[n]$  un arreglo de enteros;
2  $last\_S \leftarrow -1$ ;
3 para  $i \leftarrow 0$  a  $n - 1$  hacer
4   si  $last\_S = -1$  entonces
5      $Dist[i] \leftarrow 0$  ;
6   en otro caso
7      $Dist[i] \leftarrow i - last\_S$  ;
8   si  $T_i = S$  entonces
9      $last\_S \leftarrow i$  ;
10 devolver  $Dist$  ;
```

Algoritmo 11: PREPARE-POINTERS(n, S, A)

```

1 Sea  $R[n]$  un arreglo de enteros ;
2 Sea  $lptr[n]$  un arreglo de enteros ;
3  $l \leftarrow 0$  ;
4 mientras  $l < n$  hacer
5    $r \leftarrow l$  ;
6   mientras  $r < n$  y  $S[A[l]] = S[A[r]]$  hacer
7      $r \leftarrow r + 1$  ;
8    $lptr[r - 1] \leftarrow l$  ;
9    $R[A[r - 1]] \leftarrow r - 1$  ;
10  para  $i \leftarrow l$  a  $r - 2$  hacer
11     $R[A[i]] \leftarrow r - 1$  ;
12     $lptr[i] \leftarrow 0$  ;
13   $l \leftarrow r$  ;
14 devolver  $R, lptr$ 
```

Algoritmo 12: MOVE-TO-FRONT($R, lptr, L_j, l, r$)

```

1  para  $i \leftarrow l$  a  $r - 1$  hacer
2     $start \leftarrow L_j[i] - j$  ;
3     $pos\_bucket \leftarrow R[start]$  ;
4     $front\_bucket \leftarrow lptr[pos\_bucket]$  ;
5    si  $front\_bucket < 0$  entonces
6       $front\_bucket \leftarrow \neg front\_bucket$  ;
7     $lptr[pos\_bucket] \leftarrow \neg(front\_bucket + 1)$  ;
8    si  $front\_bucket = pos\_bucket$  entonces
9       $lptr[pos\_bucket] \leftarrow \neg lptr[pos\_bucket]$  ;
10 para  $i \leftarrow l$  a  $r - 1$  hacer
11    $start \leftarrow L_j[i] - j$  ;
12    $pos\_bucket \leftarrow R[start]$  ;
13   si  $lptr[pos\_bucket] < 0$  entonces
14      $R[start] = (\neg lptr[pos\_bucket]) - 1$  ;
15      $lptr[pos\_bucket] \leftarrow \neg lptr[pos\_bucket]$  ;
16      $lptr[R[start]] \leftarrow R[start] + 1$  ;
17   en otro caso
18      $R[start] \leftarrow lptr[pos\_bucket] - 1$  ;
19 para  $i \leftarrow l$  a  $r - 1$  hacer
20    $start \leftarrow L_j[i] - j$  ;
21    $lptr[R[start]] \leftarrow lptr[R[start]] - 1$  ;

```

Algoritmo 13: SORT-BUCKETS($R, lptr, L, m$)

```

1  para  $j \leftarrow 1$  a  $m$  hacer
2     $l \leftarrow 0$  ;
3    mientras  $l < |L_j|$  hacer
4       $r \leftarrow l$  ;
5      mientras  $r < |L_j|$  y  $S[L_j[l]] = S[L_j[r]]$  hacer
6         $r \leftarrow r + 1$  ;
7      MOVE-TO-FRONT( $R, lptr, L_j, l, r$ ) ;
8       $l \leftarrow r$  ;

```

Algoritmo 14: REASSIGN-MAPPING(X, R)

```

1   $l \leftarrow 0$  ;
2   $id \leftarrow 0$  ;
3  mientras  $l < |X|$  hacer
4     $r \leftarrow l$  ;
5    mientras  $r < |X|$  y  $R[X[l]] = R[X[r]]$  hacer
6       $r \leftarrow r + 1$  ;
7    para  $i \leftarrow l$  a  $r - 1$  hacer
8       $R[X[i]] \leftarrow id$  ;
9     $id \leftarrow id + 1$  ;
10    $l \leftarrow r$  ;

```

Finalmente, el siguiente algoritmo construye la cadena reducida tomando como referencia a las

posiciones con sufijos de tipo S.

Algoritmo 15: BUILD-REDUCED-STRING-WITH-S(n, S, T)

```

1   $A \leftarrow \text{GROUP-BY-CHARACTER}(n, S)$  ;
2   $Dist \leftarrow \text{GET-S-DISTANCES}(n, S, T)$  ;
3   $m \leftarrow \max_{i=0}^{n-1} \{Dist[i]\}$  ;
4  Sean  $L_1 \dots L_m$  listas vacías de enteros ;
5  para  $i \leftarrow 0$  a  $n - 1$  hacer
6  |    $idx \leftarrow A[i]$  ;
7  |   si  $Dist[idx] > 0$  entonces
8  |   |   Agregar  $idx$  al final de  $L_{Dist[idx]}$  ;
9   $R, lptr \leftarrow \text{PREPARE-POINTERS}(n, S, A)$  ;
10  $\text{SORT-BUCKETS}(R, lptr, L, m)$  ;
11  $X \leftarrow \{0 \leq i < n : T_i = S\}$  ;
12 Ordenar  $X$  por  $R$  creciente ;
13  $\text{REASSIGN-MAPPING}(X, R)$  ;
14  $S' \leftarrow \varepsilon$  ;
15 para  $i \leftarrow 0$  a  $n - 1$  hacer
16 |   si  $T_i = S$  entonces
17 |   |    $S' \leftarrow S' + R[i]$  ;
18 devolver  $S'$ 
```

Es sencillo notar cómo se debería realizar la implementación para ordenar las subcadenas de tipo L en función al algoritmo para las subcadenas de tipo S.

Finalmente, el algoritmo sería el siguiente:

Algoritmo 16: SAKA(n, S)

```

1 si  $n = 1$  entonces
2   devolver  $\{0\}$  ;
3 en otro caso
4    $T \leftarrow \text{CLASSIFY}(n, S)$  ;
5    $\text{cnt}_L \leftarrow |\{0 \leq i < n - 1 : T_i = L\}|$  ;
6    $\text{cnt}_S \leftarrow n - 1 - \text{cnt}_L$  ;
7   si  $\text{cnt}_L < \text{cnt}_S$  entonces
8      $S' \leftarrow \text{BUILD-REDUCED-STRING-WITH-L}(n, S, T)$  ;
9     si  $S'$  tiene todos sus caracteres diferentes entonces
10       $X \leftarrow \text{Calcular } X \text{ en funci3n al caracter de cada posici3n}$  ;
11     en otro caso
12       $X \leftarrow \text{SAKA}(\text{cnt}_L, S')$  ;
13      $A \leftarrow \text{INDUCE-ORDER-WITH-L}(n, S, T, X)$  ;
14     devolver  $A$  ;
15   en otro caso
16      $S' \leftarrow \text{BUILD-REDUCED-STRING-WITH-S}(n, S, T)$  ;
17     si  $S'$  tiene todos sus caracteres diferentes entonces
18       $X \leftarrow \text{Calcular } X \text{ en funci3n al caracter de cada posici3n}$  ;
19     en otro caso
20       $X \leftarrow \text{SAKA}(\text{cnt}_S, S')$  ;
21      $A \leftarrow \text{INDUCE-ORDER-WITH-S}(n, S, T, X)$  ;
22     devolver  $A$  ;

```

Este algoritmo tiene importancia debido a que sienta las bases para el algoritmo SAIS de Nong *et al.*.

2.4. Nong *et al.*

En [4], Nong *et al.* proponen una definici3n nueva sobre las presentadas por Ko y Aluru, los sufijos LMS:

Definici3n 15 (Sufijo LMS)

Se define un sufijo LMS como un sufijo $\text{Suf}(S, i)$ que es de tipo S bajo la condici3n de que $\text{Suf}(S, i - 1)$ exista y sea de tipo L.

Con esta definici3n podemos intentar predecir hacia donde se quiere llegar. Consideremos el orden de los sufijos en el suffix array final y analicemos qu3 pasa con un sufijo S_i de tipo L:

- S_{i+1} es de tipo S: S_{i+1} es un LMS.
- S_{i+1} es de tipo L: S_{i+1} es un sufijo que aparece **antes** que S_i en el suffix array.

Es sencillo notar que si podemos obtener el orden de los sufijos LMS, es posible inducir el orden de los sufijos de tipo L iterando de atr3s hacia adelante en el suffix array y con toda esa informaci3n podemos inducir el orden de los sufijos de tipo S.

Entonces, el objetivo que se plantea es ordenar los sufijos LMS; para ello, se definen las subcadenas LMS:

Definición 16 (Subcadena LMS)

Se define una subcadena LMS como una subcadena $S[i, j]$ tal que se cumpla alguna de las siguientes condiciones:

- $i < j$ y tanto $\text{Suf}(S, i)$ como $\text{Suf}(S, j)$ son sufijos LMS y no existe un índice $i < k < j$ tal que $\text{Suf}(S, k)$ sea un sufijo de tipo LMS.
- $S[i, j] = \text{Suf}(S, n - 1)$.

En la primera condición, el carácter $S[j]$ es llamado *overlapping character*, ya que está compartido entre dos subcadenas LMS.

Denotaremos el conjunto de posiciones LMS como $\mathcal{LMS}$.

Ejemplo 17

Considerando la cadena “MMIISSIIPPII\$”, los tipos de sus sufijos son los siguientes:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$S[i]$	M	M	I	I	S	S	I	I	S	S	I	I	P	P	I	I	\$
$T[i]$	L	L	S	S	L	L	S	S	L	L	S	S	L	L	L	L	S
$\mathcal{LMS}$			•				•				•						•

Las subcadenas LMS son:

- “IISSI”
- “IISSI”
- “IIPPII\$”
- “\$”

El siguiente lema establece una propiedad importante sobre la cantidad de sufijos LMS en una cadena.

Lema 18

Dada una cadena S de longitud n , la cantidad de sufijos LMS en S no excede a $\lceil \frac{n-1}{2} \rceil$.

A diferencia del SAKA, el algoritmo SAIS no necesita seleccionar un tipo en específico de sufijos; en vez de ello, usa los sufijos LMS para simplificar la idea.

Teorema 19 (Nong et al., 2009)

Todos los sufijos de tipo L pueden ser ordenados si el orden de los sufijos LMS es conocido.

El algoritmo 18 se puede usar para inducir el orden de todos los sufijos de S usando el arreglo X como el conjunto $\mathcal{LMS}$ ordenado por los sufijos correspondientes a sus posiciones.

Para ordenar los sufijos LMS podemos obtener una cadena reducida como en el algoritmo SAKA tomando las subcadenas LMS. El orden establecido para comparar las subcadenas LMS es el siguiente debido al Corolario 8.

Enfoque 20

Dos subcadenas LMS X y Y deben ser comparadas lexicográficamente usando el par $(\text{caracter}, \text{tipo})$ en cada posición, siendo el tipo L menor lexicográficamente que S.

Algoritmo 17: COMPUTE-BUCKETS(n, S, end)

```

1 Sea  $cnt[0 \dots (|\Sigma| - 1)]$  un arreglo de enteros inicializado con 0 ;
2 para  $c \in S$  hacer
3    $cnt[c] \leftarrow cnt[c] + 1$  ;
4 Sea  $P[0 \dots (|\Sigma| - 1)]$  un arreglo de enteros ;
5  $sum \leftarrow 0$  ;
6 para  $i \leftarrow 0$  a  $|\Sigma| - 1$  hacer
7   si  $end = \text{FALSE}$  entonces
8      $P[i] \leftarrow sum$  ;
9      $sum \leftarrow sum + cnt[i]$  ;
10  en otro caso
11     $sum \leftarrow sum + cnt[i]$  ;
12     $P[i] \leftarrow sum - 1$  ;
13 devolver  $P$ 

```

Ejemplo 21

Consideremos la cadena $S = \text{"AABACACBACBCBCB\$"}$, los tipos de los sufijos son los siguientes:

<i>Index</i>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
<i>S</i>	A	A	B	A	C	A	C	B	A	C	B	C	B	C	B	\$
<i>Type</i>	S	S	L	S	L	S	L	L	S	L	S	L	S	L	L	S
<i>LMS</i>				•		•			•		•		•			•

Las subcadenas LMS ordenadas se acuerdo al Enfoque 20 son:

0. "\$"
1. "ACA"
2. "ACBA"
3. "ACB"
4. "BCB\$"
5. "BCB"

Notemos que "ACBA" debe recibir un mapeo menor que "ACB" debido a que el caracter 'B' en la primera es de tipo L mientras que el que está en la segunda es de tipo S. Finalmente, la cadena reducida es:

$$R = 123540.$$

El siguiente teorema nos señala cómo es posible ordenar las subcadenas LMS usando el algoritmo INDUCE-ORDER.

Teorema 22 (Nong *et al.*, 2009)

Es posible ordenar las subcadenas LMS según el Enfoque 20 llamando a INDUCE-ORDER($n, S, T, \mathcal{LMS}^{char}$), donde $\mathcal{LMS}^{char}$ es $\mathcal{LMS}$ ordenado por el caracter en cada posición correspondiente.

Algoritmo 18: INDUCE-ORDER(n, S, T, X)

```

1 Sea  $A[0 \dots (n-1)]$  un arreglo de enteros inicializado en 0 ;
2  $R \leftarrow \text{COMPUTE-BUCKETS}(n, S, \text{TRUE})$  ;
3 para  $i \leftarrow |X| - 1$  a 0 hacer
4    $c \leftarrow S[X[i]]$  ;
5    $A[R[c]] \leftarrow X[i]$  ;
6    $R[c] \leftarrow R[c] - 1$  ;
7  $L \leftarrow \text{COMPUTE-BUCKETS}(n, S, \text{FALSE})$  ;
8 para  $i \leftarrow 0$  a  $n - 1$  hacer
9   si  $A[i] > 0$  and  $T[A[i] - 1] = L$  entonces
10     $c \leftarrow S[A[i] - 1]$  ;
11     $A[L[c]] \leftarrow A[i] - 1$  ;
12     $L[c] \leftarrow L[c] + 1$  ;
13  $R \leftarrow \text{COMPUTE-BUCKETS}(n, S, \text{TRUE})$  ;
14 para  $i \leftarrow n - 1$  a 0 hacer
15   si  $A[i] > 0$  and  $T[A[i] - 1] = S$  entonces
16     $c \leftarrow S[A[i] - 1]$  ;
17     $A[R[c]] \leftarrow A[i] - 1$  ;
18     $R[c] \leftarrow R[c] - 1$  ;
19 devolver  $A$  ;

```

Luego de que las subcadenas LMS estén ordenadas, ellas deben recibir su mapeo para la cadena reducida. El Algoritmo 19 describe como calcular el mapeo en tiempo lineal, donde $X = \mathcal{LMS}^{\text{mapping}}$, donde $\mathcal{LMS}^{\text{mapping}}$ es $\mathcal{LMS}$ ordenado según el Enfoque 20.

Algoritmo 19: COMPUTE-MAPPING(n, S, T, X)

```

1 Sea  $\sigma : [0, n-1] \mapsto [0, n-1]$  un mapeo vacío ;
2  $\sigma(X[0]) \leftarrow 0$  ;
3  $names \leftarrow 1$  ;
4 para  $k \leftarrow 1$  a  $|X| - 1$  hacer
5    $i \leftarrow X[k-1]$  ;
6    $j \leftarrow X[k]$  ;
7    $L \leftarrow 0$  ;
8   mientras  $i + L < n$  y  $j + L < n$  y  $(S[i+L], T[i+L]) = (S[j+L], T[j+L])$ 
9     hacer
10    si  $len > 0$  entonces
11      si  $i + L$  es LMS o  $j + L$  es LMS entonces
12        break ;
13     $L \leftarrow L + 1$  ;
14   si  $i + L$  es LMS y  $j + L$  es LMS entonces
15      $\sigma(X[k]) \leftarrow \sigma(X[k-1])$  ;
16   en otro caso
17      $\sigma(X[k]) \leftarrow \sigma(X[k-1]) + 1$  ;
18    $names \leftarrow \sigma(X[i]) + 1$  ;
19 devolver  $\sigma, names$ 

```

Ya que cada caracter de la cadena es escaneado a lo mucho dos veces, la complejidad del algoritmo

es $O(n)$.

El siguiente teorema garantiza la correctitud del algoritmo SAIS.

Teorema 23 (Nong *et al.*, 2009)

Es posible ordenar los sufijos de S llamando a $\text{INDUCE-ORDER}(n, S, T, \mathcal{LMS}^{\text{suffix}})$, donde $\mathcal{LMS}^{\text{suffix}}$ es $\mathcal{LMS}$ ordenado por el sufijo de cada posición correspondiente.

Para calcular $\mathcal{LMS}^{\text{suffix}}$, primero construiremos la cadena reducida R y luego usaremos recursión para determinar su suffix array. Sin embargo, si los caracteres de R son diferentes, la recursión no es necesaria, ya que podemos calcular el suffix array de R directamente por los caracteres (ya que habrá solo 1 posición por cada uno). Luego de calcular el suffix array de R , se debe remapear las posiciones LMS a sus equivalentes en S .

En resumen, el Algoritmo 20 describe el proceso completo usado por el algoritmo SAIS.

Algoritmo 20: SAIS(n, S)

```

1   $T \leftarrow \text{COMPUTE-TYPES}(n, S)$  ;
2   $P \leftarrow \{i > 0 : T[i-1] = \text{L} \wedge T[i] = \text{S}\}$  ;
3  Ordenar  $P$  por el primer caracter  $S[P[i]]$  ;
4   $A \leftarrow \text{INDUCE-ORDER}(n, S, T, P)$  ;
5  Ordenar  $P$  según  $A$  ;
6   $\sigma, \text{names} \leftarrow \text{COMPUTE-MAPPING}(n, S, T, P)$  ;
7  Ordenar  $P$  en orden creciente ;
8  Sea  $R$  una secuencia de tamaño  $|P|$  ;
9  para  $i \leftarrow 0$  a  $|P| - 1$  hacer
10    $R[i] \leftarrow \sigma(P[i])$  ;
11 si  $\text{names} < |P|$  entonces
12    $SA_1 \leftarrow \text{SAIS}(|R|, R)$  ;
13 en otro caso
14   Calcular  $SA_1$  como la permutación inversa de  $R$  ;
15  $SA_1 \leftarrow \{P[SA_1[i]] : i = 0, \dots, |SA_1| - 1\}$  ;
16  $SA \leftarrow \text{INDUCE-ORDER}(n, S, T, SA_1)$  ;
17 devolver  $SA$ 
```

Notemos que la línea 3 puede ser calculada en $O(n)$ usando COUNTING-SORT, y las líneas 5 y 7 pueden ser ejecutadas en una sola iteración filtrando las posiciones LMS. Además, ya que σ mapea enteros en el rango $[0, n-1]$, puede ser almacenado en un arreglo de tamaño n . En consecuencia, el algoritmo satisface $|R| \leq \left\lceil \frac{|S|-1}{2} \right\rceil$ gracias al Lema 18. Por lo tanto, la complejidad satisface la siguiente recurrencia:

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \Rightarrow T(n) = O(n)$$

Referencias

- [1] Juha Kärkkäinen, Peter Sanders, and Stefan Burkhardt. Linear work suffix array construction. *Journal of the ACM (JACM)*, 53(6):918–936, 2006.
- [2] Pang Ko and Srinivas Aluru. Space efficient linear time construction of suffix arrays. In *Annual Symposium on Combinatorial Pattern Matching*, pages 200–210. Springer, 2003.

- [3] Udi Manber and Gene Myers. Suffix arrays: a new method for on-line string searches. *siam Journal on Computing*, 22(5):935–948, 1993.
- [4] Ge Nong, Sen Zhang, and Wai Hong Chan. Linear suffix array construction by almost pure induced-sorting. In *2009 data compression conference*, pages 193–202. IEEE, 2009.